Chapter 8. Metaprogramming

This chapter covers

- Learning about literate CoffeeScript
- Constructing domain-specific languages
- Writing programs that write programs
- Being metacircular: CoffeeScript in CoffeeScript

The term *metaprogramming* is often used to refer to any programming technique sufficiently complicated that you should think thrice before using it. This chapter is *not* about writing complicated programs. Metaprogramming is also often used to refer to the use of metaobjects, where you create objects that create objects. This chapter is not about programming metaobjects—you were already working with metaobjects back in chapter 5. So, then, what is this chapter about?

The most succinct description of metaprogramming is *programs that write programs*. That's what this chapter is about. More importantly, this chapter is about changing the way you think about your programs and the language you write them in. To begin thinking differently, you'll start by swapping programs with program explanations through *literate CoffeeScript*. Next, you'll explore the *creation* of miniprogramming languages as domain-specific languages (DSLs). Finally, you'll look at programs that write programs and at how you can change the language you work in by using the CoffeeScript compiler from inside a CoffeeScript program.

8.1. Literate CoffeeScript

In literate programming, as first explained by Donald Knuth,[1] the visible structure of the program source isn't the structure of the executable program. Instead, the visible structure of the program is an explanation of the program in another language. Interspersed code snippets make up the executable program. Inspired by the idea of literate programming, the